



User Manual

Makerspace Information & Notification Display

Senior Project Spring 2026

Created by:

Anna Noto, Matthew Bernardino, & Jake Tandy

Table of Contents

Part 1 - User Guide.....	3
Overview.....	4
Quick Start Guide.....	5
Updating the Display Content.....	6
Troubleshooting (Non-Technical).....	8
Part 2 - Technical Documentation.....	10
System Overview.....	11
System Architecture.....	11
Hardware.....	11
Software.....	14
Power.....	15
Installation & Setup.....	16
Code Structure.....	19
Configuration.....	19
Maintenance Procedures.....	20
Troubleshooting (Technical).....	22
Testing & Validation.....	25
Improper display symptoms.....	25
Improper syncing symptoms.....	26
Check power output.....	27
Additional Notes.....	29
Future upgrades.....	29
Part 3 - Optional Add-On.....	30
Overview: Touchscreen Controller.....	31
Core Components.....	31
System Characteristics.....	31
Hardware Setup.....	31
Battery Maintenance.....	32
Software Setup.....	33
Running the Application.....	33
Customizing the Controller.....	34
Troubleshooting.....	36
References.....	39
Appendix.....	39
Acknowledgements.....	40

Part 1 – User Guide

Overview

The Makerspace Information & Notification Display is a centralized, real-time digital signage system designed to improve communication within the Makerspace environment. The system uses a multi-panel LED matrix display controlled by a Raspberry Pi to present clear, continuously updated information to students, staff, and faculty.

The display is designed to be visible from a distance and operate continuously during Makerspace hours with minimal user intervention.

Displayed Information Includes

- Current Makerspace shop status (OPEN / CLOSED)
- Announcements and time-sensitive updates
- Equipment or resource availability
- General informational messages

Content is dynamically pulled from a shared Google Sheet and updated automatically at regular intervals.

User Responsibilities

- Updating and maintaining display content via the Google Sheet
- Using the physical switch to accurately reflect Makerspace status
- Ensuring the system remains powered and unobstructed
- Reporting or escalating issues when necessary

No technical background is required for normal operation.

Quick Start Guide

1. Powering the System On:

1. Turn on the main power supply for the LED panels via 120VAC wall outlet plug.
2. Power on the Raspberry Pi if it is not already connected to the power 120VAC wall outlet plug.
3. Allow the system a short time to boot and initialize.

2. What should happen on startup:

1. The Raspberry Pi begins its boot process and loads the display program.
2. The software initializes the LED matrix configuration and prepares the panels for output.
3. The program reads the current display content from the configured source and sends it to the LED matrix.
4. After startup is complete, the display should begin showing the message from the google sheets specified cells.

3. What normal operation looks like:

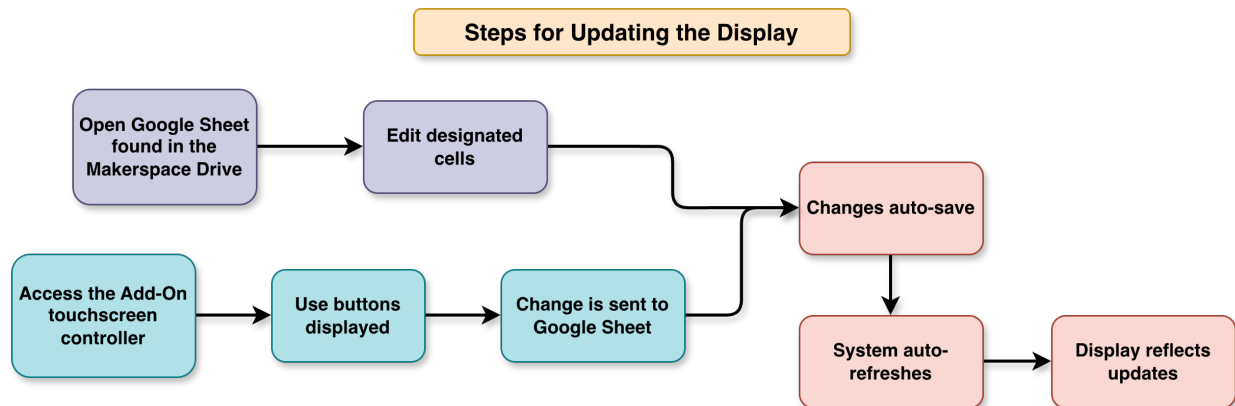
- The LED panels and Raspberry Pi remain powered and the display is stable and updating with information from the google sheet.
- The displayed message matches the google sheets cell information given a short time to update if recently inputting new information.
- The system continues to run without user interaction

4. Powering the system off:

Typically the system will always have power due to the building always being on, unless there is maintenance needed.

1. Shut down the Raspberry Pi properly before removing power via the switch on the power supply.
2. Disconnect the power supply from the 120VAC outlet.
3. Do not remove cables from panel to panel as this is communication or tamper with power supply to panel connections.

Updating the Display Content



How to access the google sheet:

1. Open assigned Google Sheet using the shared link.
2. Edit only the cells for the display to update.

What each section controls:

There will be three primary sections: Top, Middle, and Bottom.

- a. Top will control displaying to the top row of 6 panels for information.
- b. The middle will control displaying the middle 6 panels for information.
- c. The Bottom will control displaying the bottom 6 panels for information.
 - Typically it will be for status of workshop 'Open or Closed', Daily announcement, and status of items like drills/3D printers.

How long updates take to appear:

- Changes made to the Google Sheets will not appear instantly on the display.
- There is a refresh cycle that will be checked if the Google Sheet has new information to update.
- Given the set interval, the display will update with new information.

Formatting Rules:

- Keep messages short so they can fit on the display.
- Avoid long sentences or large blocks of text.
- Use simple words that can be read quickly.
- Avoid extra spacing, symbols, that can cause the display to not correctly display.

Controller (add-on):

- This is an additional touchscreen to update the display.
- It is simple and intended for quick preset changes to the display.
- See **Part 3** of this manual for more information.

Troubleshooting (Non-Technical)

Always allow sufficient startup time before troubleshooting and restart the system if necessary.

What to do if the display is blank

Quick Checks:

1. Ensure the power supply is plugged in and switched on.
2. Confirm the Adafruit Triple LED Matrix Bonnet is properly seated on the Raspberry Pi GPIO header.
3. Check that the ribbon cable from the bonnet is connected to the DATA IN port of the first panel.
4. Verify that all panels are oriented correctly (DATA OUT » DATA IN throughout the chain).

Visual Cue:

- If all panels are completely dark, the system is likely not receiving power.
- If the Raspberry Pi is on but panels are off, there may be a loose connection at the bonnet or first panel.

Tip:

Start troubleshooting from the beginning of the chain (power » bonnet » first panel). If the first panel does not receive data, none of the chained panels will turn on.

What to do if the WiFi is not working

Quick Checks:

1. Confirm the Raspberry Pi is powered on.
2. Verify the system is within range of a known WiFi network.
3. Restart the system using the power switch.
4. Ensure the correct network credentials were entered during setup.

Visual Cue:

- The display may remain static or show old content if WiFi is disconnected.
- No visible changes over time typically indicates loss of network connection.

Tip:

If WiFi issues persist, contact Roy Torres from Pacific Technology for assistance

What to do if the content is not updating**Quick Checks:**

1. Confirm the device is connected to WiFi.
2. Verify that updates were saved correctly in the Google Sheet.
3. Check that formatting rules (character limits, structure) were followed.
4. Wait at least one full refresh cycle for changes to appear.

Visual Cue:

- The display shows content, but it does not reflect recent changes made in the sheet.
- Some sections may update while others remain unchanged if formatting is incorrect.

Tip:

If content still does not update, try making a small test change (ex: edit a single word) to confirm the system is still syncing properly.

What to do if the panels are partially off**Quick Checks:**

1. Make sure all ribbon cables connecting the panels are fully seated. Ribbon cables daisy-chain from the bonnet to the first panel, then to the next, and so on.
2. Ensure the Adafruit Triple LED Matrix Bonnet is correctly connected to the Raspberry Pi.
3. Check that the power supply is on and delivering power.

Visual Cue:

- If a panel is completely dark, it may not be receiving power or data.
- If panels flicker or show partial content, reseat the ribbon cables starting from the bonnet.

Tip:

Panels can be replaced individually if reseating does not solve the issue.

Part 2 - Technical Documentation

System Overview

The system is a modular display platform integrating hardware and software components to deliver real-time visual information.

Core Components

- **Raspberry Pi 4B** – system controller and processing unit
- **Adafruit RGB Matrix Bonnet** – signal interface between controller and display
- **LED Matrix Panels** – visual output system
- **Power Distribution System** – regulated power delivery network
- **Software Stack** – data acquisition, processing, and rendering

System Characteristics

- Continuous operation capability
- Modular and scalable architecture
- Remote content management
- Serviceable and maintainable design

System Architecture

Hardware

The system hardware is designed as a modular, scalable platform for driving a multi-panel LED matrix display. It consists of a centralized control unit, a dedicated signal interface, a daisy-chained display array, and a structured mounting system.

Control and Interface

Raspberry Pi 4B (Control Unit)

The Raspberry Pi serves as the primary processing and control unit for the system.

- Executes display control software
- Manages data retrieval and processing
- Provides GPIO interface for hardware communication

- Supports continuous, unattended operation

Adafruit RGB Matrix Bonnet (Signal Interface)

The Adafruit RGB Matrix Bonnet provides the electrical and logical interface between the Raspberry Pi and the LED matrix panels.

- Directly mounts to Raspberry Pi GPIO header
- Converts GPIO signals into HUB75-compatible outputs
- Handles signal timing and multiplexing for LED control
- Provides ribbon cable connectors for panel input

This component is required for proper communication between the Raspberry Pi and the display panels.

Display System

LED Matrix Panels

The display consists of multiple 64×64 RGB LED panels configured as a single logical display.

- Resolution: 64×64 pixels per panel
- Pixel pitch: 3 mm
- Viewing angle: $\geq 160^\circ$
- Configuration: Daisy-chained via HUB75 interface

Panels are connected sequentially, allowing synchronized rendering across the full display surface.

Panel Configuration

- Panels are arranged in a multi-row grid
- Data flows from the bonnet to the first panel, then propagates through the chain
- System mapping aligns physical layout with software-defined coordinates

Mechanical Structure

Mounting Frame

The display is supported by an aluminum extrusion frame designed for rigidity and modularity.

- Material: 2020/2040 aluminum extrusion
- Supports scalable panel expansion
- Provides structural alignment and load distribution

Panel Mounting

- Panels are secured using machine screws, spacers, and brackets
- Custom 3D-printed brackets provide additional support and alignment
- Spacing ensures proper airflow and cable routing

Design Characteristics

Modular: Panels and components can be added or replaced independently

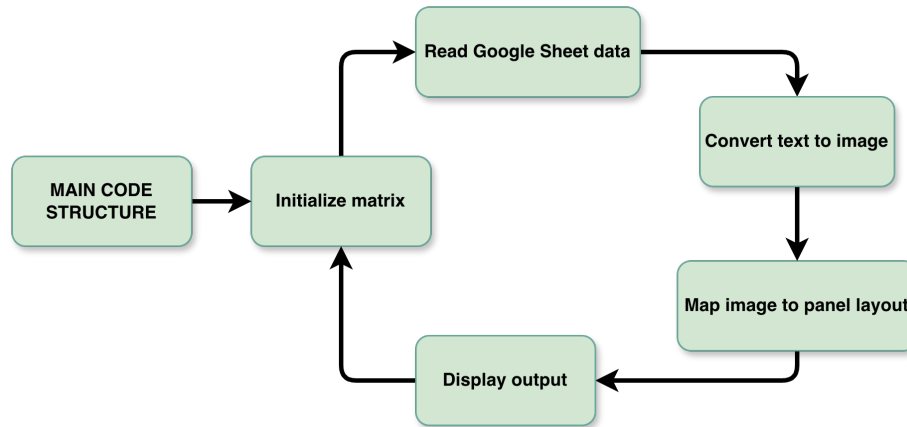
Scalable: System supports expansion to larger display configurations

Serviceable: Components are accessible for maintenance and replacement

Thermally Managed: Open frame and active cooling support sustained operation

Software

The software connects the Google Sheet data to the physical LED Matrix display in an easy intuitive way to display information.



Work Flow

Google Sheets

- Main editor and user interface.
- Store the text to display led matrix panels.

Gspread

- Connect python to the Google Sheets.
- Authenticates access and reads cell data from the Google Sheet.

Pillow

- Converts the text from spreadsheet and renders it into image format.
- Pixel-based image that makes text from the spreadsheet.

LED Matrix

- Given pixel data from the Raspberry Pi.
- Displays the final image that is text from the Google Sheet on the panels.

Program Loop Behavior

1. User updates specified cells in Google Sheets.
2. Python program accesses the sheet through gspread.

3. Spreadsheet data is read into the program.
4. Pillow formats the text into an image/pixel format.
5. The LED matrix library sends the pixel data to the display hardware.
6. The content will appear on the LED Matrix Display.
7. This will loop over a set time to check for new information on the google sheet and update the display.
8. If content is different it will update the display via the pipeline; otherwise, it will not detect a change and not update the display waiting for the next refresh to check again.

Power

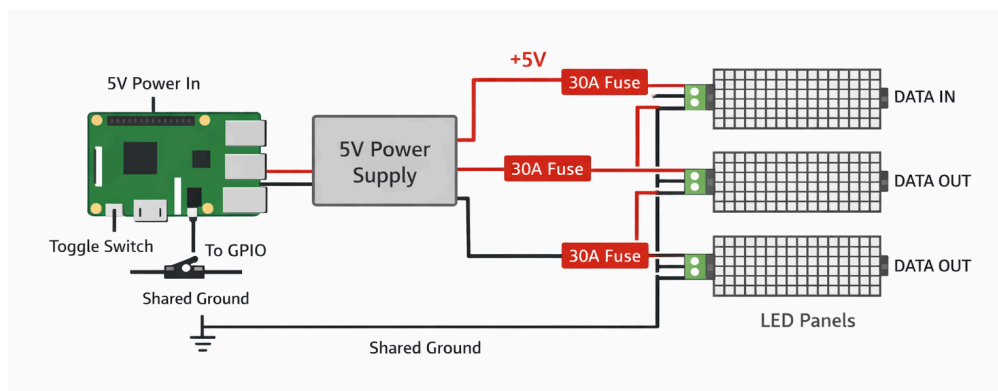
Power supply (Mean Well)

- Converts 120V AC directly from wall outlet to 5V for distribution
- 5V distribution throughout the whole circuit, matches demand of each panel
- 3 +V and -V terminals for distribution to the panels

Bus bars, fuses, wiring

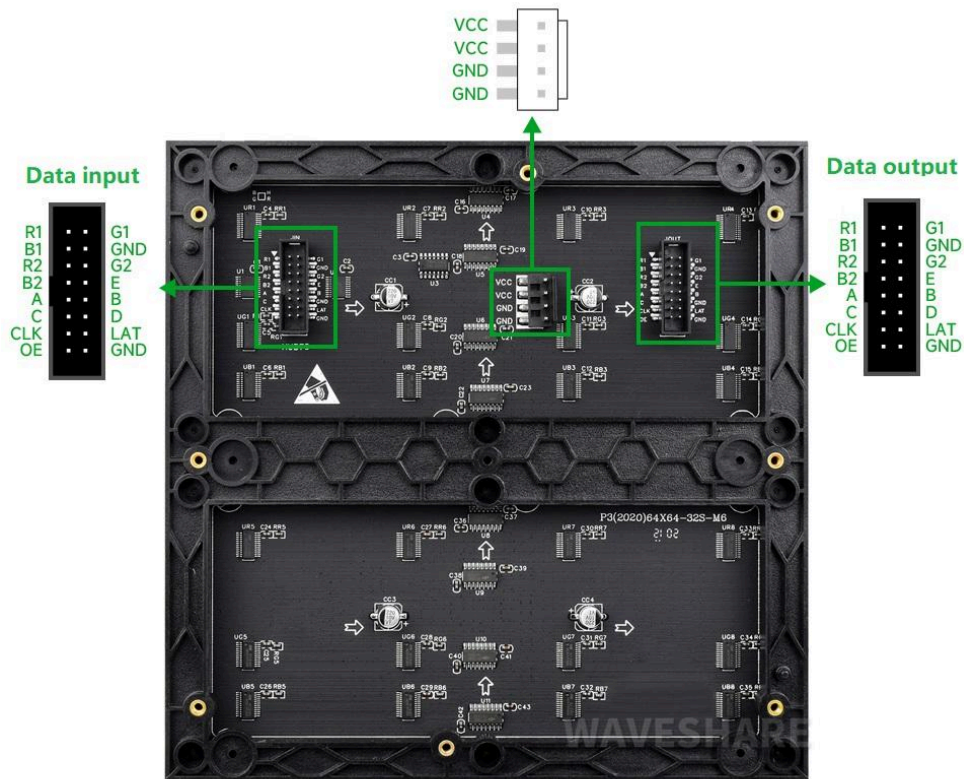
- 30A fuses utilized, prevents any shorts in the circuit
- Fuses connected directly to the power supply, first thing in series with the power supply
- Terminal blocks utilized to separate out the distribution of power, 3 terminals for the 3 rows of panels
- Lever connectors utilized to ensure proper connections and keep wiring organized

Power distribution design



Installation & Setup

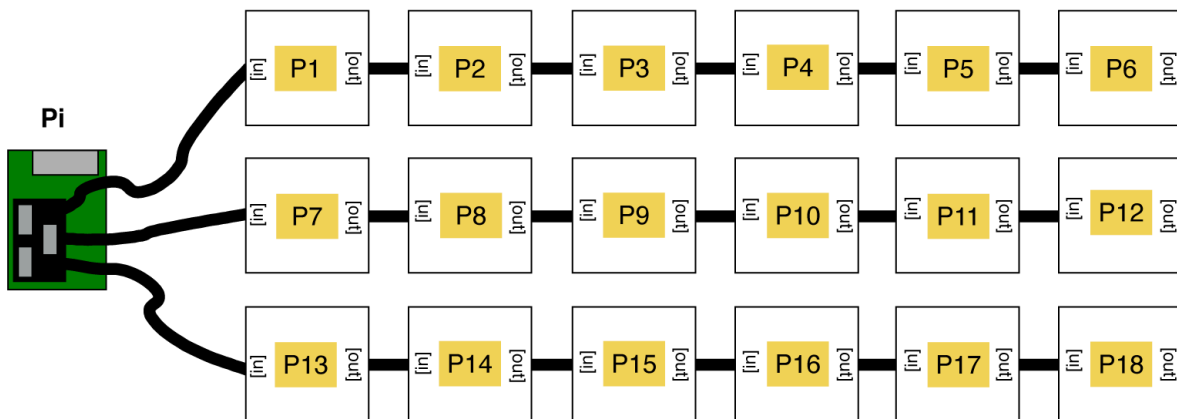
Hardware Assembly



1. Mount all 64×64 LED panels onto the frame, ensuring that all edges are aligned and panels sit flush with one another.
2. Verify panel orientation before wiring:
 - a. Each panel has a DATA IN and DATA OUT port.
 - b. Panels must be oriented so that the data flows correctly from one panel to the next.
 - c. Incorrect orientation will prevent downstream panels from displaying properly.
3. Connect ribbon cables from the Adafruit Triple LED Matrix Bonnet to the DATA IN port of the first panel of each row.
4. Continue daisy-chaining panels in each row by connecting:
 - a. DATA OUT (Panel N) » DATA IN (Panel N+1)

- b. The last panel of each row should not have anything connected to the DATA OUT port.
5. Attach the bonnet securely to the Raspberry Pi using the GPIO header.
6. Confirm all panels are securely fastened and all ribbon cables are fully seated.

Wiring Overview (when looking at the back of the display)



Power System

The power system is made up of 3 main components:

1. Power Supply

- First component in the circuit
- Wall outlet connects in this orientation: Line, Neutral, Ground, (Black, White, Green)
- 3 +V and -V terminals for distribution to the panels
- Check to see if the green light next to the +V terminals is on, to see if the power supply is operating correctly

2. Fuse safety

- Fuses connected directly to the power supply, first thing in series with the power supply
- Check fuse if a row or whole display are not working

- Use continuity test to check fuse operation

3. Secure wiring

- Wiring is connected properly and safely in an organized manner, using lever connectors and terminals to secure connections
- If improper display occurs and fuse is fine, check for any breaks in wire or any loose connections.

Raspberry Pi

1. Raspberry Pi 4B

- Main controller for the system.
- Runs the program that reads Google Sheets and sends the data to the LED matrix panels to display.

2. OS Installation

1. Install Raspberry Pi OS Trixie 64bit on microSD card
2. Insert MicroSD card into raspberry pi and complete initial setup
3. Setup internet connection
4. Disable audio so no interference with adafruit HAT

3. Required Libraries

Install the following Python libraries for the project

- gspread
- Pillow
- rpi-rgb-led-matrix

4. Google API Key

1. Using a Google account, setup a service account key file for authentication
 - This key allows the Raspberry Pi to securely access the project Google Sheets
2. The JSON key file must be stored in the correct project directory
3. Running The Program
 - Create an environment using the terminal

- This will house all the libraries and python scripts
- Run the python script that controls the display
 - During normal use the program will remain running so the display stays up to date with information.

Code Structure

1. File Organization

- a. The project files will remain under an organized directory “led-matrix”

2. Key Scripts

a. led_matrix.py

- Runs the full program
- Connects google sheets, processing text, converting text, and displaying the LED matrix display.

b. Setup Files

- The libraries installed will be here as well as the LED Matrix native library with its setup files.
- Credential files will also be stored here so the gspread library can authenticate with the key provided from the Google service account.
- Additional files for touch panel integration and setup.

Configuration

1. Change Display Layout

- Update layout values in the script
- Change panel rows, columns, and display size
- Generally do not make changes as these changes to the program can break or cause unintended effects on the display.

2. Modify Refresh Rate

Set a refresh variable in seconds (Ex: refresh_time = 300 (in seconds))

- Lower value = faster updates
- Higher value = slower updates

3. Update Google Sheet Structure

Change the cell range to be read in the script (Ex: `c_range = "B1:B3"`)

- Updates the cells that will be read and displayed on the LED matrix panels

4. Adjust Panel Mapping

Change `chain_length` and `parallel` to match the physical setup of the LED matrix panels.

- Ex: `p_height = 192` (Total panel height size of 64 pixels * 3 panels tall)
- Ex: `p_width = 384` (Total panel width size of 64 pixels * 6 panels wide)

Maintenance Procedures

Checking Connections

- Periodically inspect ribbon cables and bonnet seating.
- Ensure panels are not loose or misaligned.

Replacing Panels

Note: Because of the daisy-chain setup, replacing one panel does not require disconnecting the entire chain.

1. Turn off the power supply.
2. Disconnect ribbon cables from the faulty panel.
3. Remove the panel from the frame.
4. Install the new panel, reconnect the ribbon cable, and verify seating.
5. Power on the system and run a test pattern.

Monitoring the Power System

Ensuring Proper Enclosure

- Periodically check that the enclosure for the power system is secure
- Ensure that the power components are still within the enclosure
- Ensure that there is no dust buildup inside of the enclosure

Ensuring Proper Mounting

- Ensure that the enclosure and power components are not pressed firmly against the wall
- Ensure that power supply is firmly in position within the enclosure

Ensure Proper Function Of Power Components

- Test continuity across the fuses
- Test output of the power supply
- Test end of line wires output

Cleaning & Cooling Considerations

Check Enclosure Behavior

- Check for dust buildup
 - If seen, safely remove the enclosure from the board and use wipes to clean the affected area
- Check for any signs of overheating

Test The Cooling Components

- Take the enclosure off and let it run for >10 minutes
 - Observe the system and check the temperature of the key components
 - If cool to the touch, then place enclosure back on
 - If warm to the touch, replace the cooling system components.

Troubleshooting (Technical)

GPIO Issues

If the Raspberry Pi boots normally but the display does not respond, the GPIO connection may be the cause.

What to Check:

- Power down the system completely before inspecting GPIO connections.
- Confirm the Adafruit Triple LED Matrix Bonnet is fully seated on the Raspberry Pi GPIO header.
- Inspect the GPIO pins for any bent, misaligned, or damaged pins.
- Ensure no wires or components are interfering with the bonnet connection.
- Verify the bonnet is installed in the correct orientation.

How to Test:

- Reseat the bonnet carefully and restart the system.
- If available, test with a known working bonnet or Raspberry Pi.

Visual Cue:

- Raspberry Pi powers on, but panels stay dark or never initialize.
- Display may briefly flash during startup and then turn off.

Tip:

Most GPIO issues are caused by loose seating or pin misalignment. Never force the bonnet onto the header.

Panel Not Responding in Chain

If one or more panels in the chain do not display content, the issue is usually related to data flow or power between panels.

What to Check:

- Confirm ribbon cables run in the correct direction: DATA OUT → DATA IN.
- Ensure every ribbon cable is firmly connected.
- Check that power cables are connected to each affected panel if applicable.

- Verify panel addresses/order in the software configuration matches the physical chain layout.
- Inspect the non-working panel for visible damage.

How to Test:

- Swap the suspected panel with a working panel in the chain.
- Connect the non-working panel directly as the first panel to determine if it functions independently.
- Replace ribbon cables one at a time to rule out cable failure.

Visual Cue:

- First panel works, later panels stay dark.
- One middle panel is blank while others continue working.
- Content appears cut off where the failed panel begins.

Tip:

If one panel fails early in the chain, all panels after it may also stop receiving data.

API / Authentication Errors

If the display is powered and connected to WiFi but content does not update, the system may be unable to access external services such as Google Sheets or APIs.

What to Check:

- Confirm the Raspberry Pi has internet access.
- Verify API credentials, service account files, or authentication tokens are present in the correct directory.
- Ensure the Google Sheet has not been renamed, deleted, or had permissions changed.
- Confirm sharing permissions allow the configured account to access the file.
- Check whether API quotas or request limits have been reached.

How to Test:

- Restart the software or reboot the Raspberry Pi.
- Reauthenticate the service account if credentials expired or changed.
- Open the Google Sheet on another device to confirm it is online and accessible.

Visual Cue:

- Display continues showing old data indefinitely.
- No updates occur despite successful edits to the sheet.
- Error messages may appear in terminal logs if accessed directly.

Tip:

Most authentication issues happen after credentials are moved, renamed, or sharing permissions are changed.

Rendering Issues

If content updates but appears distorted, cut off, overlapping, or incorrectly positioned, the problem is likely related to rendering settings or formatting.

What to Check:

- Verify text stays within recommended character limits.
- Check for unsupported symbols or unusual formatting in the Google Sheet.
- Confirm panel resolution and total display dimensions match the software configuration.
- Ensure fonts used by the software are installed correctly.
- Verify brightness and refresh settings are within stable ranges.

How to Test:

- Replace long text with a short test phrase.
- Remove special characters or emojis.
- Restart the software after configuration changes.
- Test with default font/settings if custom settings were added.

Visual Cue:

- Text overlaps or scrolls incorrectly.
- Graphics appear shifted or cropped.
- Flickering, ghosting, or unstable brightness.

Tip:

Most rendering issues are caused by content formatting or incorrect display resolution settings rather than hardware failure.

Testing & Validation

Improper display symptoms

Verifying Proper display:

Use this procedure to confirm that panels, data, and power are functioning correctly

Improper display symptoms include:

- Panels light up inconsistently
- Colors appear incorrect
- Some panels remain dark
- Flickering

Checks:

- **Confirm the controller is powered**
 - Verify the controller's 5V input is active.
 - Ensure the controller shares ground with all panels.
- **Enable built-in test patterns**
 - Use the controller's software interface or hardware test mode.
 - Look for solid colors (red/green/blue), gradients, and chase patterns.
- **Verify uniform brightness**
 - All panels should display identical intensity during solid-color tests.
 - Dim or uneven panels may indicate voltage drop or loose power connections.
- **Check power distribution**
 - Measure voltage at the first and last panel in each row (target: 4.8–5.1V under load).

- **Inspect HUB75 ribbon cables**

- Ensure ribbons are fully inserted and oriented correctly (pin-1 alignment).
- Replace any visibly damaged cables.

Improper syncing symptoms

Verify panel sync:

Use this procedure to confirm that all panels remain refreshed in sync and display frames in the intended order.

Improper syncing symptoms include:

- Horizontal tearing
- Panels updating at different times
- One row “lags” behind the others
- Partial frames/Improper alignment

Checks

1. Confirm correct panel order

- Ensure panels are chained left-to-right within each row.
- Verify the controller output goes to Row 1 » Row 2 » Row 3 (or your configured order).

2. Run a physical test

- Use a patterning test.
- All panels should update simultaneously.

3. Check grounding

- A missing or floating ground can cause sync issues.
- Ensure the Shared Ground ties together:

- PSU negative
- All panel rows
- Controller ground

4. Verify data cable integrity

- HUB75 cables longer than ~1 meter may cause timing issues.
- Replace long or low-quality ribbons.

Check power output

Use this procedure to confirm that the 5V power system is delivering stable, sufficient current to all LED panels and the controller.

Symptoms:

- Panels dim or flicker
- Some panels reboot or flash randomly
- Colors shift or appear washed out
- Controller resets intermittently
- Voltage drop across rows

General Checks:

1. Verify PSU Output Voltage

- Measure directly at the **Mean Well LRS-600-5** output terminals.
- Expected reading: **5.0V ± 0.1V** under load.
- If voltage is low:
 - Check AC input connection
 - Ensure PSU fan is running
 - Reduce load or inspect for short circuits

2. Confirm Main 30A Fuse Integrity

- Inspect the **30A ATO blade fuse** immediately after the PSU.
- Replace if blown or heat-discolored.
- A blown main fuse indicates:
 - Overcurrent condition
 - Shorted panel or wiring fault

3. Check Voltage at the Terminal Bus Bars

- Measure voltage at the bus bars feeding the three branches.
- Expected: **4.9–5.1V**
- If significantly lower than PSU output:
 - Inspect wiring gauge (should be 10–12 AWG)
 - Shorten cable runs
 - Tighten terminal screws

4. Test Each Branch Output

For each of the three panel rows:

1. Measure voltage at the **first panel** in the row
2. Measure voltage at the **last panel** in the row

Acceptable range:

- First panel: **4.8–5.1V**
- Last panel: **≥4.7V**

If the last panel is below 4.7V:

- Add thicker wire
- Reduce branch length

7. Confirm Shared Ground Integrity

- Measure continuity between:
 - PSU negative
 - All panel grounds
 - Controller ground
- A floating or intermittent ground can cause:
 - Flicker
 - Sync issues
 - Controller resets

Additional Notes**Future upgrades**

Upgrade to a Raspberry Pi 5 - Although not necessary, upgrading would lead to faster response times

Known limitations of the system

- Updates are dependent on stable network connectivity
- Performance and speed is constrained by controller hardware
- There is a fixed resolution which is based on panel configuration

Part 3 - Optional Add-On

Overview: Touchscreen Controller

The touchscreen controller is an optional add-on that provides a dedicated physical interface for updating display content without directly accessing the Google Sheet. It is intended for use by Makerspace staff who need to make quick updates to tool availability, machine shop status, or other display content during operation. The controller is designed to operate wirelessly using a 12V lithium battery and buck converter, allowing it to be used anywhere within WiFi range without needing a wall outlet. It communicates with the system by writing updated values directly to the Google Sheet, which the display system then picks up through its normal refresh cycle. It does not interface directly with the LED display hardware.

Core Components

- **Raspberry Pi 3B** – dedicated controller unit
- **Elecrow 5" LCD Touchscreen (Model B)** – touch input and visual interface
- **Kivy application** – graphical user interface software
- **12V 10000mAh lithium battery** – portable power source
- **DROK buck converter** – steps 12V down to 5V for the Raspberry Pi

System Characteristics

- Operates independently from the main display Raspberry Pi
- Wireless operation — no wall outlet required during normal use
- Requires WiFi connectivity to write to Google Sheets
- Auto-polls Google Sheet every 15 seconds to stay in sync with any direct sheet edits
- Runs as a fullscreen kiosk application on boot
- Estimated battery runtime of 6–7 hours per full charge under normal use

Hardware Setup

1. Before first use, use a multimeter to confirm the buck converter output is set to exactly 5.0V. Do not connect the Raspberry Pi until this has been verified. Voltage above 5.1V can permanently damage the Pi.

2. Connect the buck converter output to the Raspberry Pi via the 5V and GND GPIO pins (pins 2 and 6) or via a wired USB connection to the Pi's micro-USB power port.
3. Connect the 12V lithium battery to the buck converter input terminals.
4. Connect the Elecrow touchscreen to the Raspberry Pi via HDMI for display output.
5. Connect the touchscreen to the Raspberry Pi via the USB port labeled Touch on the screen for touch input.
6. Note that the touchscreen and the Raspberry Pi should not share available current with other high-draw USB peripherals. If a low voltage warning appears, disconnect any unnecessary USB devices such as a mouse or keyboard before connecting the touch cable.

Battery Maintenance

- Recharge the battery before each use or when the low voltage warning appears on screen.
- A fully charged battery provides approximately 6–7 hours of runtime under normal operating conditions.
- Do not allow the battery to fully discharge repeatedly, as this can reduce long-term battery capacity.
- Store the battery in a cool, dry location when not in use.

Software Setup

The controller runs a Python application built with the Kivy framework. The following dependencies are required:

```
kivy
gspread
google-auth
```

Install with:

```
bashpip install kivy gspread google-auth
```

The application is configured through two values at the top of **makerspace_controller.py**:

1. **SERVICE_ACCOUNT_FILE** – path to the Google service account JSON key file
2. **SHEET_ID** – the ID of the Google Sheet, found in the sheet's URL

The service account JSON file must be present in the same directory as the application and the service account must have Editor access to the Google Sheet.

Running the Application

```
cd /path/to/project
source venv/bin/activate
python3 makerspace_controller.py
```

To configure the application to launch automatically on boot, a systemd service file is recommended. Refer to the setup documentation included with the project files for the full service configuration.

Customizing the Controller

All display labels, tracked values, and Google Sheet cell references are defined at the top of **makerspace_controller.py** and can be modified without changing the core application logic.

Changing What a Button Tracks

The **CELL_MAP** dictionary maps each controller element to a specific cell in the Google Sheet.

To change what a button reads from or writes to, update the corresponding cell address:

```
CELL_MAP = {
    "drills":      "B3",
    "calipers":    "B4",
    "glue":        "B5",
    "printer_open": "E3",
    "printer_queue": "E4",
    "machine_shop": "B7",
    "machine_reopen": "B8",
    "hours_session1": "D8",
    "hours_session2": "E8",
    "projects":    "B11",
}
```

For example, if the Makerspace begins tracking soldering irons instead of glue, update **"B5"** to whichever cell in the Google Sheet now holds that value, and update the button label as described below.

Changing a Button Label

Button labels are set when each tile is created in the **MakerspaceRoot** class. Locate the following block and update the first argument of any **CounterTile** to change what it displays:

```
self.drills_tile    = CounterTile("Available\nDrills",    "drills",
... )
self.calipers_tile  = CounterTile("Available\nCalipers",  "calipers",
... )
self.glue_tile      = CounterTile("Available\nGlue",      "glue",
... )
self.printer_tile   = CounterTile("Available\n3D Printers", "printer_open",
... )
self.queue_tile     = CounterTile("3D Prints\nin Queue",  "printer_queue",
... )
```

The **\n** in the label text creates a line break. Keep labels short because two short lines display better than one long line on the tile.

Adding or Removing a Counter Tile

Row 1 supports up to four tiles and Row 2 supports up to two counter tiles alongside the machine shop and hours tiles.

To add a new counter tile, add a new entry to **CELL_MAP**, create a new **CounterTile** instance, and add it to the appropriate row layout with **row.add_widget()**. To remove a tile, delete its **add_widget** line. The remaining tiles will automatically fill the space.

Changing the Hours Preset Options

The hours chips are read-only on the controller and are updated by selecting from a dropdown in the Google Sheet.

To change the available preset options, go to the Google Sheet, click cell **D8** or **E8**, and navigate to Data ➤ Data Validation to edit the dropdown list. No changes to the controller code are required.

Changing the Auto-Refresh Interval

The controller polls the Google Sheet every 15 seconds by default. To change this, update the following value near the top of the file:

```
POLL_INTERVAL = 15 #seconds between auto-refresh from sheet
```

Note: Setting this value too low may cause the application to hit Google Sheets API rate limits. A minimum of 10 seconds is recommended.

Troubleshooting

What to do if touch input is not responding

Quick Checks:

1. Confirm the USB cable is connected from the port labeled Touch on the screen to a USB port on the Raspberry Pi.
2. Verify the battery has sufficient charge and the Pi is fully powered on.
3. Disconnect any mouse or keyboard connected to the Pi, then unplug and reconnect the touch USB cable. Run `ls /dev/input/` in the terminal.
4. A new event device should appear when the touch cable is connected. If none appears, try the other Touch port on the screen.

Visual Cue:

- If the screen displays an image but taps have no effect, the touch USB cable is likely missing or disconnected.
- If a new event device does not appear after connecting either USB port, the cable may be charge-only with no data lines » try a different cable.

Tip:

The Elecrow touchscreen requires two separate connections:

1. HDMI for the picture
2. USB for touch input

Having one without the other is the most common setup issue.

What to do if a low voltage warning appears

Quick Checks:

1. Disconnect any USB peripherals such as a mouse or keyboard – these are not needed during normal operation.
2. Check the battery charge level and recharge if low.
3. Confirm the buck converter output is still set to 5.0V using a multimeter.
4. Inspect the connection between the buck converter output and the Raspberry Pi for loose or intermittent contact.

Visual Cue:

- A lightning bolt icon in the top right corner of the screen indicates insufficient voltage.
- If the warning only appears when the touch cable is plugged in, the battery charge is likely too low to support the combined load.

Tip:

The low voltage warning can appear either because the battery is depleted or because too many devices are drawing current at once. Always disconnect unnecessary USB peripherals before connecting the touch cable, and keep the battery charged before each use.

What to do if the controller cannot connect to Google Sheets

Quick Checks:

1. Confirm the Raspberry Pi is connected to WiFi. Run `iwconfig` in the terminal to verify.
2. Verify that `service_account.json` is present in the same directory as `makerspace_controller.py`.
3. Confirm the Google Sheet has been shared with the service account email address with Editor access.
4. Confirm that `SHEET_ID` in `makerspace_controller.py` matches the ID in the Google Sheet URL.

Visual Cue:

- The status bar at the bottom of the controller will display "Connection failed – retrying..." if the sheet cannot be reached.
- If values on the controller do not change after pressing buttons, writes to the sheet are likely failing silently – check the terminal output for error messages.

Tip:

The most common causes of connection failure are an expired or missing service account JSON file, the sheet not being shared with the service account, or the Pi losing its WiFi connection. Address these in order before assuming a software issue.

What to do if button presses are not reflected on the LED display**Quick Checks:**

1. Confirm the controller is connected to Google Sheets » check the status bar for the last sync time.
2. Wait up to 15 seconds for the LED display system's next poll cycle to pick up the change.
3. Press the Refresh button on the controller to manually pull the latest values and confirm the sheet was updated.
4. Open the Google Sheet directly and verify the correct cells were updated with the expected values.

Visual Cue:

- If the controller's status bar shows a recent sync time but the LED display has not updated, the display system's poll interval has not elapsed yet – this is normal.
- If the Google Sheet cell values did not change after a button press, the write failed and the connection troubleshooting steps above should be followed.

Tip:

The controller and the LED display operate independently and communicate only through the Google Sheet. If the sheet was updated correctly, the display will reflect the change within its next poll cycle without any further action required.

References

- [1] “Raspberry Pi Documentation – Raspberry Pi OS,” *www.raspberrypi.com*.
<https://www.raspberrypi.com/documentation/computers/os.html#introduction>
- [2] J. DeVito, “DIY LED Video Wall,” *Adafruit Learning System*, Jul. 18, 2013.
<https://learn.adafruit.com/adafruit-diy-led-video-wall/frame>
- [3] H. Zeller, “Controlling RGB LED display with Raspberry Pi GPIO,” *GitHub*, May 02, 2022. <https://github.com/hzeller/rpi-rgb-led-matrix>
- [4] Pillow, “Pillow — Pillow (PIL Fork) 6.2.1 Documentation,” *Readthedocs.io*, 2011.
<https://pillow.readthedocs.io/en/stable/>
- [5] “gsread — gsread 4.0.0 documentation,” *docs.gsread.org*.
<https://docs.gsread.org/en/latest/>
- [6] “RGB-Matrix-P3-64x64 - Waveshare Wiki,” *Waveshare.com*, 2026.
https://www.waveshare.com/wiki/RGB-Matrix-P3-64x64?srsId=AfmBOooP9ToonTlO-AioTcWiz8VNCweENDFXWGxgCYbYEakWZocUmq_r
- [7] Kivy Organization, “Kivy: Cross-platform Python Framework for NUI Development,” *Kivy Documentation*, 2024.
<https://kivy.org/doc/stable/>

Appendix

Original code files for the Kivy based controller can be found here:

<https://github.com/annanoto/LED-Display-Kivy-Based-Controller>

Original code files for the google sheet system can be found here:

<https://github.com/Matthew01156/LED-Matrix-Display>

Acknowledgements

Dr. Don Lee

Mark Foreman

Jeremy Hanlon

Dr. Vivek Pallipuram

Roy Torres

The School of Engineering and Computer Science